

NYC Parking Violations FY2023 - FY2025

Big Data Lab Cluster FS26 – Projektarbeit

Autoren: Dario Meyer, Fabio Roos, Martina Gander | Abgabe: 31.05.2026

Abstract

Dieses Projekt analysiert die öffentlichen «Parking Violations Issued»-Daten der Stadt New York für die Fiskaljahre 2023 – 2025 [1]. Die Rohdaten (drei CSV-Exporte mit zusammen rund 54 Mio. Zeilen) wurden in ein verteiltes HDFS geladen, mit Apache Spark / PySpark bereinigt und als nach Fiskaljahr partitioniertes, Snappy-komprimiertes Parquet abgelegt. Der bereinigte Datensatz umfasst 49'967'563 Einträge. Beantwortet werden zwei Fragen: die Häufigkeit und Entwicklung der Verstosstypen sowie die zeitlichen Muster der Verstösse. Verstösse gegen das Tempolimit in Schulzonen dominieren klar und gehen über die drei Jahre zurück; zudem zeigen sich ausgeprägte Muster nach Wochentag und Tageszeit.

1 Introduction

Ziel des Projekts ist die Analyse einer grossen Datenmenge mit Big-Data-Werkzeugen. Als Grundlage dienen die Parking-Violation-Daten der Stadt New York [1], die mit rund 50 Mio. bereinigten Einträgen ein echtes Big-Data-Szenario darstellen. Gespeichert werden die Daten in HDFS, verarbeitet mit Spark SQL und der PySpark-DataFrame-API. HDFS ist als verteiltes Dateisystem für grosse Datenmengen ausgelegt, während Spark strukturierte Daten im Arbeitsspeicher und verteilt über mehrere Knoten verarbeitet [2], [3].

1.1 Fragestellungen

- **Frage 1:** Welche Parking-Violation-Typen kommen in FY2023 - FY2025 am häufigsten vor, und wie verändern sie sich über die Fiskaljahre?
- **Frage 2:** Gibt es zeitliche Muster bei Parking Violations nach Monat, Wochentag und Tageszeit?

2 Cluster

Für die Analyse wurde ein verteilter Hadoop-Cluster mit drei Knoten in Master-Slave-Architektur aufgesetzt: ein Master- und zwei Slave-Knoten.

Rolle	Hostname
Master	bdlc-012.bdlc.ls.eee.intern
Slave 1	bdlc-005.bdlc.ls.eee.intern
Slave 2	bdlc-018.bdlc.ls.eee.intern

2.1 Service-Aufteilung

Master (bdlc-012): NameNode (HDFS-Metadaten), Secondary NameNode (Checkpoints des Edit-Logs, kein automatisches Failover), DataNode, Spark Master (Koordination, Job- und Executor-Verteilung), Spark Worker (nutzt die Master-Ressourcen mit), Spark History Server sowie JupyterLab.

Slaves (bdlc-005, bdlc-018): je ein DataNode und ein Spark Worker. Die Web-UIs (NameNode :9870, Spark Master :8080, History Server :18080, JupyterLab :8888/lab) laufen alle auf dem Master und dienen der Überwachung und Entwicklung.

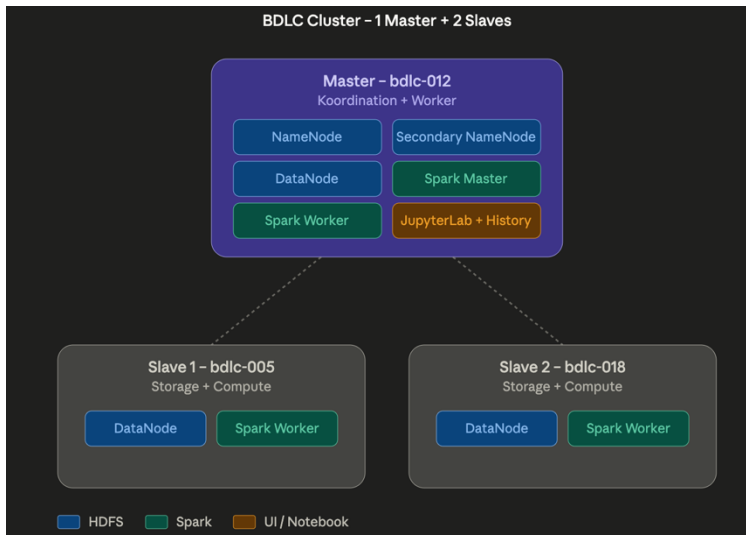
2.2 Eingesetzte Frameworks und Tools

HDFS dient als Speicher-Layer für Roh- und Parquet-Daten. Der NameNode hält die Metadaten, die DataNodes die Blöcke (Standardgrösse 128 MB). Eine Anpassung der Blocksize war nicht nötig, da die Files deutlich grösser als 128 MB sind und so sinnvoll auf mehrere Knoten verteilt werden.

Apache Spark übernimmt im Standalone-Modus die gesamte Verarbeitung. Der Spark Master koordiniert die drei Worker, die parallel rechnen. Spark wurde gegenüber MapReduce gewählt, weil es Daten im Arbeitsspeicher hält und iterative Analysen deutlich schneller erlaubt. Die Interaktion erfolgt über PySpark in JupyterLab, der zentralen Entwicklungsumgebung.

Apache Parquet ist das Speicherformat der verarbeiteten Daten, ein spaltenorientiertes Open-Source-Format. Die Snappy-Komprimierung reduziert das Volumen stark (im Projekt von 9.4 GB CSV auf 1.2 GB Parquet, rund 87 %), das Schema wird mitgespeichert, und Spark liest gezielt einzelne Spalten von der Disk statt jeden Datensatz vollständig zu deserialisieren, ein grosser Vorteil bei Aggregationen über wenige Spalten [4]. Zusätzlich werden die Daten nach Fiskaljahr partitioniert: Queries mit Jahresfilter lesen dank Partition Pruning nur die relevanten Files.

2.3 Begründung der Topologie-Wahl



Ein Master mit zwei Slaves ist die einfachste Topologie, die ein echtes verteiltes Setup darstellt. Mit drei DataNodes und Replikationsfaktor 2 ist genug Redundanz vorhanden, um den Ausfall eines Knotens ohne Datenverlust zu überstehen. Der Master übernimmt zusätzlich Worker-Aufgaben, um die Rechenressourcen voll auszunutzen. Das Set aus HDFS (Storage), Spark (Compute), Parquet (Format) und JupyterLab (Entwicklung) deckt den gesamten Big-Data-Workflow ab und entspricht der im Kurs vermittelten Standard-Architektur.

Abbildung 1 – Cluster-Topologie

2.4 Ressourcenaufteilung und Replikation

In `spark-defaults.conf` auf dem Master wurde festgelegt, wie viele Ressourcen eine Spark-Application beanspruchen darf:

```
spark.driver.memory 10g
spark.executor.memory 15g
spark.executor.cores 4
spark.cores.max 12
```

Daraus ergeben sich 12 Cores gesamt, aufgeteilt in 3 Executors à 4 Cores (ein Executor pro Worker), 45 GB Executor-Memory (3 × 15 GB) und 10 GB Driver-Memory auf dem Master. Entscheidend ist `spark.cores.max`: Jeder Worker ist über `SPARK_WORKER_CORES=12` mit 12 Cores konfiguriert, dem Cluster stehen also 36 Cores zur Verfügung. Im Standalone-Modus greift sich eine Application ohne Obergrenze alle 36 Cores, erst `spark.cores.max` begrenzt sie auf 12. Grund für die Begrenzung ist, dass beim Gemeinsamen Arbeiten nicht zu viel Cores pro Person beansprucht werden. Bei Bedarf kann jeder User, beim alleinigen Arbeiten, im JupyterLab die Cores Manuell erhöhen. Der Master erhält bewusst etwas weniger Worker-Anteil, da dort zusätzlich Driver, JupyterLab und UI-Server laufen.

Der Replikationsfaktor wurde in `hdfs-site.xml` über `dfs.replication` auf 2 gesetzt: Jeder Block liegt auf zwei DataNodes, was einen Knotenausfall abfedert, ohne den dreifachen Speicherbedarf des Defaults zu verursachen. Die Konfiguration wurde auf dem Master erstellt und per scp auf beide Slaves verteilt, damit alle Knoten identische Properties verwenden.

2.5 Funktionstests

HDFS wurde über das NameNode-UI (:9870) geprüft (alle drei DataNodes «live», korrekter Replikationsstatus) sowie mit `hdfs dfsadmin -report` und Lese-/Schreibtests. Spark wurde mit dem SparkPi-Beispiel via `spark-submit` getestet und im Master-UI (:8080) auf drei registrierte Worker kontrolliert; der History Server (:18080) zeigte die abgeschlossenen Jobs. Die erfolgreiche Verarbeitung der rund 50 Mio. Zeilen ohne Knotenausfall diente als End-to-End-Test.

3 Dataset

Verwendet werden die NYC-Open-Data-Datensätze «Parking Violations Issued» für FY2023–FY2025. NYC-Fiskaljahre laufen vom 1. Juli bis 30. Juni des Folgejahres [1]. Die Daten wurden pro Fiskaljahr als CSV vom Open-Data-Portal exportiert und unverändert ins HDFS gelegt.

Datensatz	Zeilen (roh)
FY2023	21'563'238
FY2024	16'101'101
FY2025	16'559'243
Bereinigt (gesamt)	49'967'563

Die ausgewählten Analysefelder umfassen u.a. `summons_number`, `registration_state`, `issue_date_parsed`, `violation_time`, `violation_county`, `street_name`, `vehicle_make`, `violation_code` und `violation_description_official`. Ein technischer Primary Key wurde nicht definiert; `summons_number` kann jedoch als eindeutige Ticketnummer interpretiert werden. Foreign Keys wurden nicht modelliert.

4 Prototyping Raw Data

Im Notebook `src/3_Prototyping/1.0_prototype_raw_data.ipynb` wurde ein erster Kontakt mit den Rohdaten hergestellt. Da der vollständige Datensatz ~54 Mio. Zeilen umfasst, wurde ein 1%-Sample (rund 542'000 Zeilen) gezogen, um Datenqualitätsprobleme effizient zu erkennen. Dabei wurden folgende Auffälligkeiten entdeckt: `violation_description` ist inkonsistent mit verschiedenen Formulierungen für denselben Violation Code, `violation_time` verwendet ein ungewöhnliches 12-Stunden-Format mit vereinzelt ungültigen Werten, der Datensatz enthält Tippfehler; und es wurden 403 Duplikate gefunden. Diese Erkenntnisse flossen direkt ins Pre-processing ein.

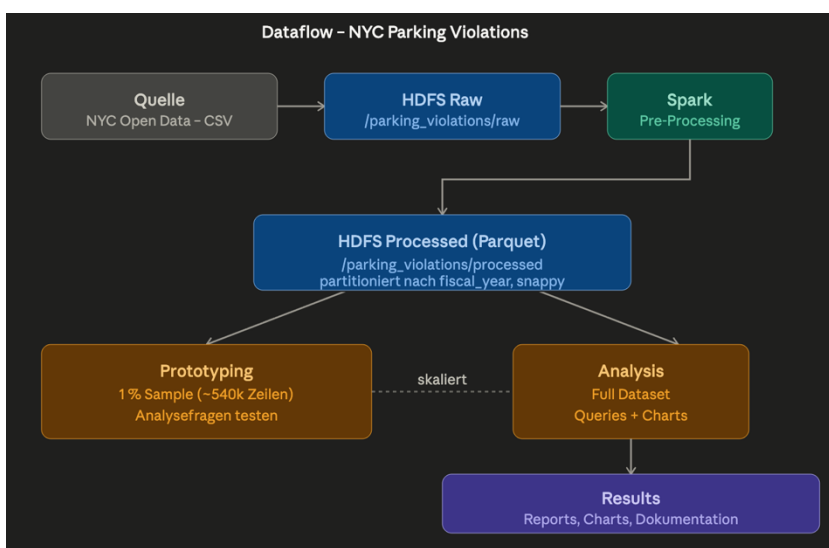
5 Pre-processing und Data Quality

Das Notebook `src/1_Pre_Processing/5.0_clean_parking_violations.ipynb` lädt die drei CSV-Rohdateien der Fiskaljahre 2023–2025 und bereitet sie auf:

- **Zusammenführen und vereinheitlichen:** Jahre mit `unionByName` zusammenführen, relevante Spalten auswählen (`summons_number`, `issue_date`, `violation_time`, `violation_code`, `violation_description`), Namen in `snake_case` vereinheitlichen.
- **Datumsfelder:** `issue_date` parsen und `issue_year`, `issue_month`, `issue_weekday` ableiten.
- **Zeitfelder:** `violation_time` bereinigen und `violation_hour`/`violation_minute` ableiten; Werte mit Stunde 0, fehlendem A/P-Suffix oder Stunde >12 werden bewusst NULL belassen.
- **Datenbereinigung:** Zeilen ohne `summons_number`, `violation_code` oder parsebares Datum entfernen; 4'250'889 Duplikate auf `summons_number` und ~2'200 Datums-Tippfehler filtern.
- **Violation Description:** Ersetzt durch `violation_description_official` aus der offiziellen Mapping-Tabelle (`ParkingViolationCodes_January2020.xlsx`).
- **Fiskaljahr-Korrektur:** Die ursprüngliche Spalte `fiscal_year` stammt nur aus dem Dateinamen. Da NYC-Fiskaljahre vom 1. Juli bis 30. Juni laufen, wird das echte Fiskaljahr (`fy`) direkt aus `issue_date_parsed` abgeleitet. Zusätzlich wird ein Fiskalmonat (`fm`, 1=Juli bis 12=Juni) eingeführt. Die Boolean-Spalte `is_complete_fy` markiert nur Datensätze der drei vollständigen Fiskaljahre FY2023–FY2025 und schliesst unvollständige Randeinträge aus.
- **Speichern:** Resultat als Snappy-Parquet nach `fy` partitioniert: `hdfs:///parking_violations/processed/parking_violations_cleaned_v5`.

Die anschliessende Prüfung zeigt: Die zentralen Analysefelder enthalten keine fehlenden Werte; `violation_hour` hat NULL-Werte für ungültige Zeitangaben, sonst ausschliesslich gültige Werte 0-23. Der Output enthält `_SUCCESS` sowie die Partitionen `fy=2023`, `fy=2024`, `fy=2025`.

6 Dataflow Dario



Die Daten durchlaufen fünf Stationen: (1) Quelle ist das NYC-Open-Data-Portal mit je einem CSV-Export pro Fiskaljahr. (2) Die CSVs werden unverändert unter `/parking_violations/raw/` als «Source of Truth» abgelegt. Bei einem Bug im Pre-Processing kann ohne erneuten Download neu gestartet werden. (3) Spark bereinigt die Daten verteilt im Cluster-Memory. (4) Das Ergebnis wird als partitioniertes Snappy-Parquet unter `/parking_violations/processed/` geschrieben. (5) Die Analyse-Notebooks lesen dieses Parquet direkt; Resultate entstehen als Charts und Tabellen im Notebook.

Abbildung 2 - Dataflow-Übersicht (Stationen 1–5)

7 Analysis Fabio

Die finalen Analysen laufen auf dem vollständigen Datensatz (49'967'563 Zeilen) in src/4_Analysis. Das Fiskaljahr wird aus dem Ausstellungsdatum abgeleitet; für zeitbezogene Auswertungen zählen nur die drei vollständig abgedeckten Fiskaljahre.

7.1 Frage 1 - Häufigste Violation-Typen

Welche Parking-Violation-Typen kommen in FY2023-FY2025 am häufigsten vor, und wie verändern sie sich über die Fiskaljahre?

Code 36 (Photo School Zone Speed Violation) dominiert mit 16,77 Mio. Verstößen über alle drei Jahre deutlich. Es folgen Code 21 (No Parking, Street Cleaning, 5,81 Mio.), Code 38 (Fail to Display Meter Receipt, 3,50 Mio.), Code 14 (No Standing Day/Time Limits, 2,50 Mio.) sowie Code 5 (Bus Lane Violation, 2,11 Mio.). Die Rangfolge bleibt über alle drei Fiskaljahre stabil.

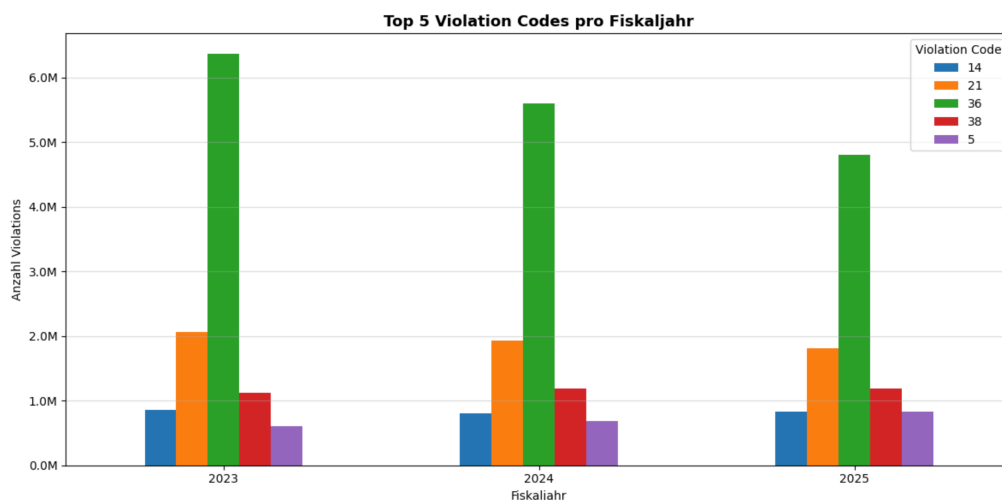


Abbildung 3 - Top-5-Violation-Codes pro Fiskaljahr, gruppiertes Balkendiagramm

Die Jahresentwicklung zeigt ein differenziertes Bild. Code 36 geht von 6,37 Mio. (FY2023) auf 4,80 Mio. (FY2025) zurück, was einem Rückgang von 24,6 % entspricht. Code 21 sinkt moderat um 11,9 %, während Codes 38 und 14 weitgehend stabil bleiben. Bemerkenswert ist der gegenläufige Anstieg bei Code 5 (Bus Lane Violation): Die Zahlen steigen von 605'189 (FY2023) auf 826'748 (FY2025), ein Zuwachs von 36,6 %.

Plausibilitätskontrolle Code 36

Code 36 wird fachlich plausibilisiert, weil er mit Abstand am häufigsten vorkommt und gleichzeitig den grössten Rückgang aufweist, beides zusammen macht ihn zum erklärungsbedürftigsten Befund. Die Analyse beschränkt sich auf Code 36, um den Rahmen des Berichts nicht zu sprengen.

Im monatlichen Chart fällt sofort der Peak im August 2022 (FY2023) auf: rund 0.77 Mio. Verstösse, deutlich über allen anderen Monaten und Fiskaljahren. Diese Auffälligkeit war der Ausgangspunkt für eine gezielte Recherche: Warum genau in diesem Monat ein so markanter Anstieg?

Die Antwort liefert eine dokumentierte Massnahme: Am 1. August 2022 begann NYC seine 2'000 Schulzonen-Kameras in 750 Schulzonen rund um die Uhr (24/7) zu betreiben, vorher liefen sie nur werktags zwischen 6:00 und 22:00 Uhr [5]. August 2022 war damit der erste Monat mit voller Kameraauslastung, während Fahrer ihr Verhalten noch nicht angepasst hatten. Die Kurve sinkt danach kontinuierlich, da die Fahrer sich vermutlich mit der Zeit an die neuen Blitzerzeiten angepasst hatten. Der monatliche Chart bestätigt diesen Befund direkt aus unseren Daten.

Der wiederkehrende Rückgang im Februar (alle drei Fiskaljahre) ist ein saisonales Muster, weniger Tage im Monat und eingeschränktes Fahraufkommen im Winter. Das übergeordnete Muster (Juli hoch -> Februar tief -> Frühling Erholung) entspricht dem allgemeinen saisonalen Fahraufkommen in NYC. Eine Analyse von NYC-Verkehrsdaten zeigt, dass der Verkehr im Sommer seinen Höchststand erreicht und im Januar/Februar auf den Jahrestiefpunkt sinkt [6]. Die Verstosszahlen spiegeln dieses Muster direkt wider. Der auffällige Einbruch am Ende von FY2025 (Juni 2025) könnte einen Reporting-Lag des zuletzt exportierten Dateimonats darstellen und bleibt eine offene Folgefrage.

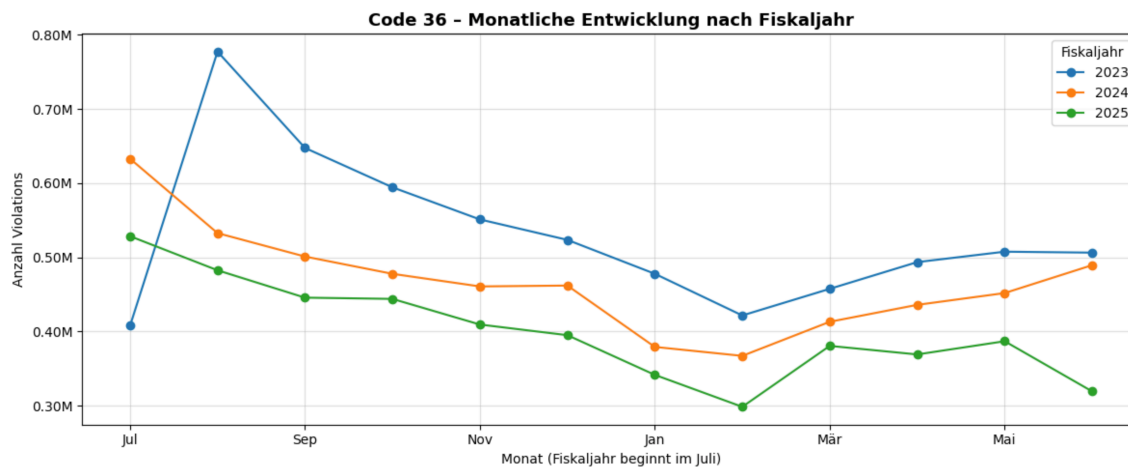


Abbildung 4 - Code 36, monatliche Entwicklung nach Fiskaljahr

Fazit

Code 36 dominiert mit 16,77 Mio. Verstößen klar. Der Rückgang von 24,6 % ist keine Datenfluktuation, sondern dokumentierte Verhaltensanpassung der Autofahrer an die 24/7-Kameraüberwachung. Bemerkenswert ist der gegenläufige Anstieg bei Bus-Lane-Verstößen (+36,6 %), der auf neue Kontrollschwerpunkte der Stadt hindeutet. Die Daten spiegeln weniger eine generelle Ab- oder Zunahme von Verstößen wider, sondern eine Verschiebung im Fokus der städtischen Verkehrskontrollen.

7.2 Frage 2 - Zeitliche Muster

Gibt es zeitliche Muster bei Parking Violations nach Monat, Wochentag und Tageszeit?

Monat: Alle drei Fiskaljahre zeigen ein konsistentes saisonales Muster: Die Violations-Zahlen sind im Sommer am höchsten und sinken im Winter auf den Jahrestiefstwert. Besonders auffällig ist der Peak im August 2022 (FY2023) mit rund 1.77M Violations, ein Wert, der direkt auf die in Analyse 1 dokumentierte Aktivierung der 24/7-Schulzonenüberwachung zurückzuführen ist.

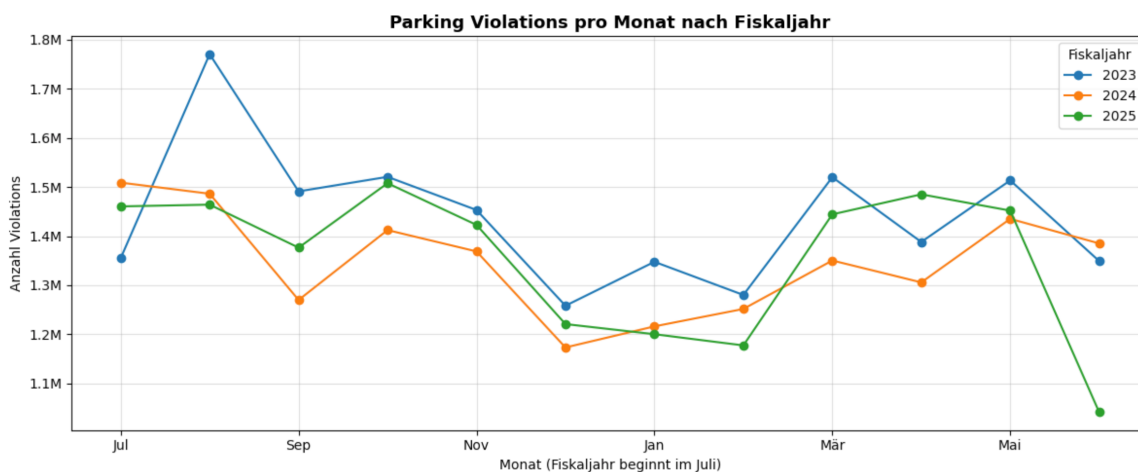


Abbildung 5 - Parking Violations nach Monat

Woche: Auf Wochentageebene zeigt sich ein klares Werktag-Wochenende-Gefälle. An Wochenenden ist generell weniger Verkehr auf den Strassen und gleichzeitig sind bestimmte Parking-Regeln ausgesetzt, was die tieferen Violations-Zahlen erklärt.

Tageszeit: Violations konzentrieren sich stark zwischen 8 und 14 Uhr, mit einer starken Abnahme nachts.

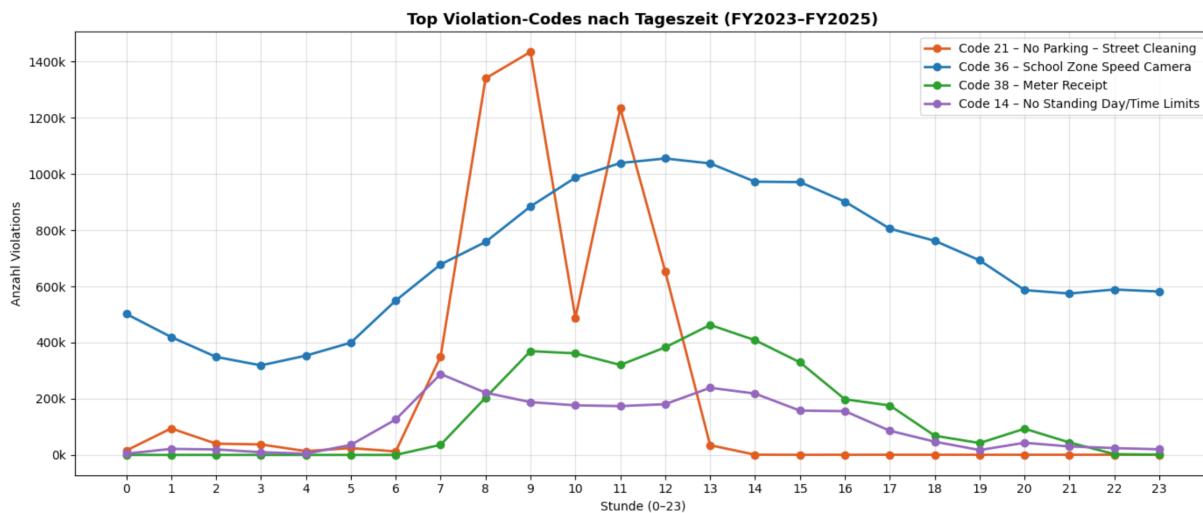


Abbildung 6 - Top 4 Parking Violations nach Tageszeit

Plausibilitätskontrolle: Die Einzelanalyse der vier häufigsten Violation-Codes nach Stunde zeigt, dass der Gesamtpeak die Überlagerung verschiedener Strafzettel-Typen ist. Code 21 (Street Cleaning, 5.81M Violations) ist fast ausschliesslich zwischen 8 und 13 Uhr aktiv, entsprechend den typischen ASP-Zeitfenster (Alternate Site Parking Regelung). Code 38 (Parkuhr) folgt einem Business-Hours-Muster mit Peak bei 13–14 Uhr, Code 14 (No Standing) zeigt Rush-Hour-Doppelpicks bei 7 und 13–14 Uhr, und Code 36 (Schulzone) bleibt als einziger Code auch nachts aktiv. Sonntags sind ASP und Parkuhren in NYC generell ausgesetzt, was den tiefsten Wert direkt erklärt.

Parking Violations in NYC folgen einem klar strukturierten zeitlichen Rhythmus, der eng mit dem städtischen Alltag verknüpft ist. An Werktagen, während der Geschäftszeiten und im Sommer werden Bussen am häufigsten ausgestellt.

Als neue Fragestellung ergäbe sich: Inwiefern verändert die zunehmende Kameraüberwachung den Tag-Nacht-Rhythmus der Violations langfristig?

8 Conclusion und Learnings

Der technische Aufbau gestaltete sich einfacher als erwartet: Das Zusammenbinden der einzelnen virtuellen Maschinen zu einem funktionierenden Cluster wird von Werkzeugen wie Hadoop und Spark weitgehend automatisch übernommen, sodass wir uns rasch auf die Datenarbeit konzentrieren konnten.

Die grössten Herausforderungen lagen in der bewussten Steuerung der Ressourcen und der Datenqualität. Die Spark-Konfiguration muss aktiv gemanagt werden: Parallele Sessions blockieren Worker-Ressourcen, weshalb Sessions konsequent mit `spark.stop()` beendet werden müssen. Hier zeigte sich auch, wie wichtig die Absprache im Team ist, da uns nur begrenzte Rechenleistung zur Verfügung stand, mussten wir bewusst koordinieren, wann paralleles Arbeiten sinnvoll ist und mit wie vielen Cores gearbeitet wird, und wann eine Person die gesamten Ressourcen benötigt.

Ein wichtiges Learning war der Wert des Prototypings. Das Arbeiten mit einem 1 %-Sample der Rohdaten hat sich bewährt, um die Daten kennenzulernen und Ansätze zu erproben. Mit dem vollständigen Datensatz wäre dieses explorative Vorgehen sehr ressourcenintensiv gewesen. Ausserdem hat es sich gelohnt, nur mit den Spalten, welche für die Fragestellungen relevanten sind, zu arbeiten. Denn obwohl Spark beim Einlesen von CSV-Dateien grundsätzlich alle Spalten parsen muss, reduziert die frühe Selektion den Speicherbedarf stark. Spark kann mehr Daten parallel im Memory halten und muss seltener auf die Disk auslagern, was das Laden der Daten beschleunigte. Es zahlt sich also aus, von Beginn an zu wissen, welche Fragen beantwortet werden sollen, und konsequent nur mit den dafür benötigten Variablen zu arbeiten.

Ein letztes, übergreifendes Learning betrifft die Zeitplanung. Bei einem Projekt dieser Art haben wir gelernt, bewusst genügend Puffer einzuplanen, da man immer wieder in «Rabbit Holes» gerät, sei es, weil etwas Technisches nicht auf Anhieb funktioniert und neue Lösungswege gefunden werden müssen, oder weil eine Auffälligkeit in den Daten genauer untersucht oder bereinigt werden muss.

Quellen

- [1] NYC Open Data: Parking Violations Issued – Fiscal Years [2023](#), [2024](#), [2025](#). data.cityofnewyork.us
- [2] Apache Hadoop Documentation: HDFS Architecture. hadoop.apache.org
- [3] Apache Spark Documentation: Spark SQL, DataFrames and Datasets Guide. spark.apache.org
- [4] Apache Spark Documentation: Parquet Files. spark.apache.org
- [5] New York City Department of Finance. (n.d.). School zone speed camera violations. NYC.gov. <https://www.nyc.gov/site/finance/vehicles/school-zone-camera-violations.page>
- [6] Gardner, D. R. (n.d.). Seasonal trends in NYC traffic: STL part I. [gardner.fyi](http://www.gardner.fyi/blog/STL-Part-I/). <http://www.gardner.fyi/blog/STL-Part-I/>